



OSTEOPOROSIS DETECTION

Using Deep Learning Techniques

MODULARIZATION OF PROBLEM

- Knee X-ray image dataset
- Class labels: Normal, Osteopenia, Osteoporosis
- Dataset inspection and class distribution analysis

MODULE 1

Data Collection & Dataset Understanding

- Image resizing and normalization
- Train-validation-test split
- Data augmentation (rotation, flip, zoom)
- Handling class imbalance

MODULE 2

Data Preprocessing & Augmentation

- Transfer Learning models (CNN-based)
- Ensemble learning for performance improvement
- Fine-tuning pre-trained networks

MODULE 3

Feature Learning & Model Development

- Metrics: Accuracy, Precision, Recall, F1-score, ROC-AUC
- Confusion matrix analysis
- Explainable AI: Grad-CAM / SHAP visualization

MODULE 4

Model Evaluation & Explainability

- Streamlit-based user interface
- Image upload and prediction
- Display class label with confidence score

MODULE 5

Application & Deployment

- Python, TensorFlow / Keras
- OpenCV, NumPy, Matplotlib
- Streamlit for deployment

COMPUTING FRAMEWORK

DESIGN METHODOLOGY

- **Data Preprocessing**
 - All images were resized to a fixed input size required by CNN models
 - Normalization was applied to scale pixel values
- **Data augmentation techniques were used:**
 - Rotation
 - Horizontal flipping
 - Zooming
- **Augmentation helped in:**
 - Reducing overfitting
 - Handling class imbalance
 - Improving generalization

Model	Architecture Type	Key Strength	Computational Cost	Performance Trend
VGG16	Deep CNN (16 layers)	Simple & stable archi	High	Good baseline accur
ResNet50	Residual Network	Handles deep learnin	Medium-High	Better than VGG
InceptionV3	Multi-scale CNN	Captures fine texture	Medium	High accuracy
VGG 19	Deep CNN (19 layers)	Rich feature extraction	High	Comparable / better tha

Table 1: Comparison of Models

VGG 16

```
train_gen = data_generator(train_paths, train_labels)
val_gen = data_generator(val_paths, val_labels)

# Define VGG16 Model
base_model = VGG16(weights='imagenet', include_top=False, input_shape=(224, 224, 3))
base_model.trainable = False

x = base_model.output
x = GlobalAveragePooling2D()(x)
x = Dense(128, activation='relu')(x)
x = Dropout(0.5)(x)
output = Dense(3, activation='softmax')(x)

model = Model(inputs=base_model.input, outputs=output)
model.compile(optimizer=Adamax(learning_rate=0.001), loss='categorical_crossentropy', metrics=['accuracy'])

# Train model
history = model.fit(
    train_gen,
    validation_data=val_gen,
    steps_per_epoch=len(train_paths) // 64,
    validation_steps=len(val_paths) // 64,
    epochs=100,
    callbacks=[tf.keras.callbacks.EarlyStopping(monitor='val_loss', patience=5, restore_best_weights=True)]
)
```

ResNet50

```
from tensorflow.keras.applications import ResNet50

# Define ResNet50 Model
base_model = ResNet50(weights='imagenet', include_top=False, input_shape=(224, 224, 3))
base_model.trainable = False

x = base_model.output
x = GlobalAveragePooling2D()(x)
x = Dense(128, activation='relu')(x)
x = Dropout(0.5)(x)
output = Dense(3, activation='softmax')(x)

model = Model(inputs=base_model.input, outputs=output)
model.compile(
    optimizer=Adamax(learning_rate=0.001),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

# Train ResNet50
history_resnet = model.fit(
    train_gen,
    validation_data=val_gen,
    steps_per_epoch=len(train_paths) // 64,
    validation_steps=len(val_paths) // 64,
    epochs=100,
    callbacks=[tf.keras.callbacks.EarlyStopping(
        monitor='val_loss', patience=5, restore_best_weights=True)]
)

# Save Model
model.save("/kaggle/working/ResNet50_knee_osteo_model.keras")
```

VGG 19

```
# Define VGG19 Model
base_model = VGG19(weights='imagenet', include_top=False, input_shape=(224, 224, 3))
base_model.trainable = False

x = base_model.output
x = GlobalAveragePooling2D()(x)
x = Dense(128, activation='relu')(x)
x = Dropout(0.5)(x)
output = Dense(3, activation='softmax')(x)

model = Model(inputs=base_model.input, outputs=output)
model.compile(
    optimizer=Adamax(learning_rate=0.001),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

# Train VGG19
history_vgg19 = model.fit(
    train_gen,
    validation_data=val_gen,
    steps_per_epoch=len(train_paths) // 64,
    validation_steps=len(val_paths) // 64,
    epochs=100,
    callbacks=[tf.keras.callbacks.EarlyStopping(
        monitor='val_loss', patience=5, restore_best_weights=True)]
)

# Save Model
model.save("/kaggle/working/VGG19_knee_osteo_model.keras")
print("VGG19 training complete!")
```

InceptionV3

```
from tensorflow.keras.applications import InceptionV3

# Define InceptionV3 Model
base_model = InceptionV3(weights='imagenet', include_top=False, input_shape=(224, 224, 3))
base_model.trainable = False

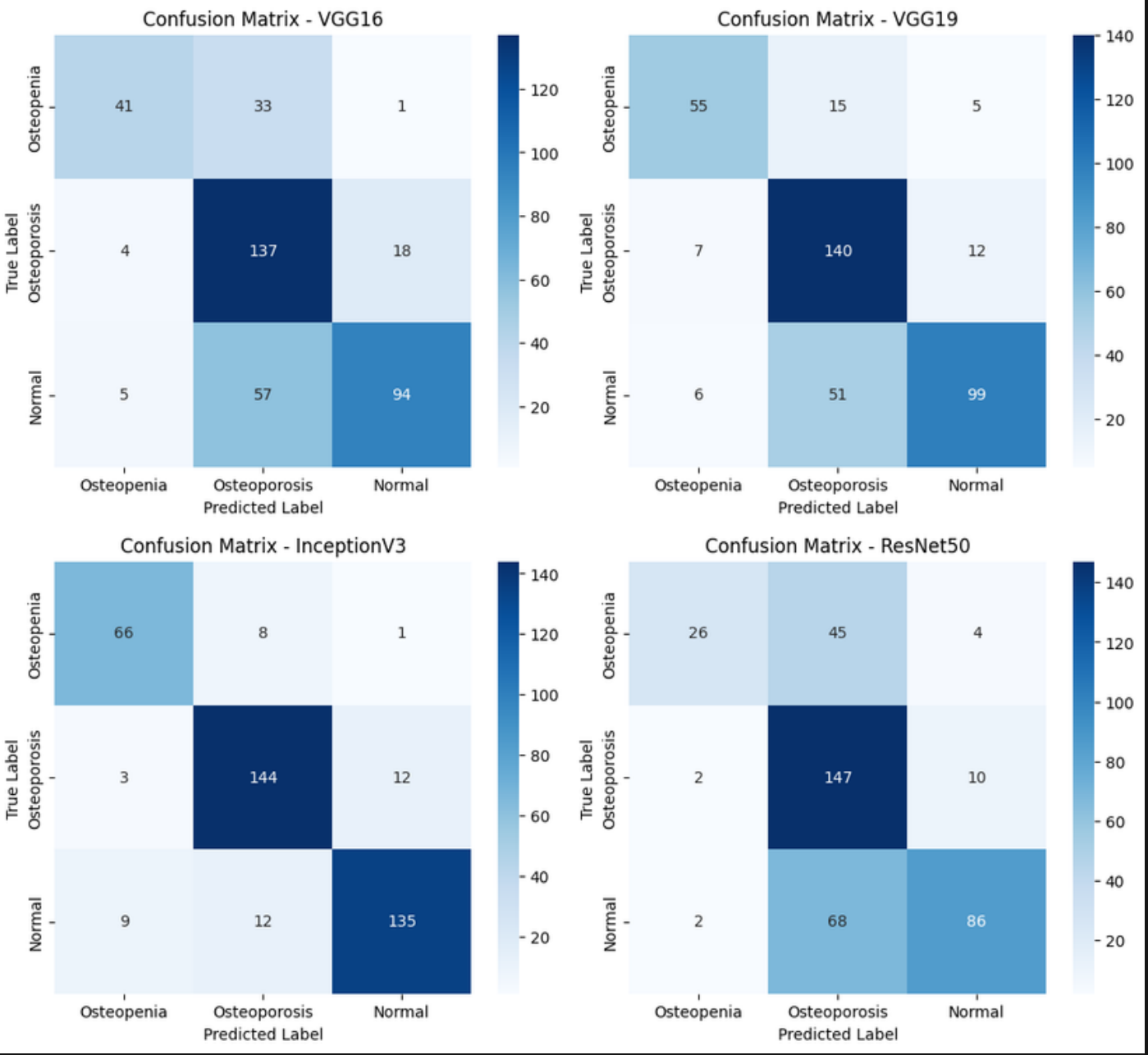
x = base_model.output
x = GlobalAveragePooling2D()(x)
x = Dense(128, activation='relu')(x)
x = Dropout(0.5)(x)
output = Dense(3, activation='softmax')(x)

model = Model(inputs=base_model.input, outputs=output)
model.compile(
    optimizer=Adamax(learning_rate=0.001),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

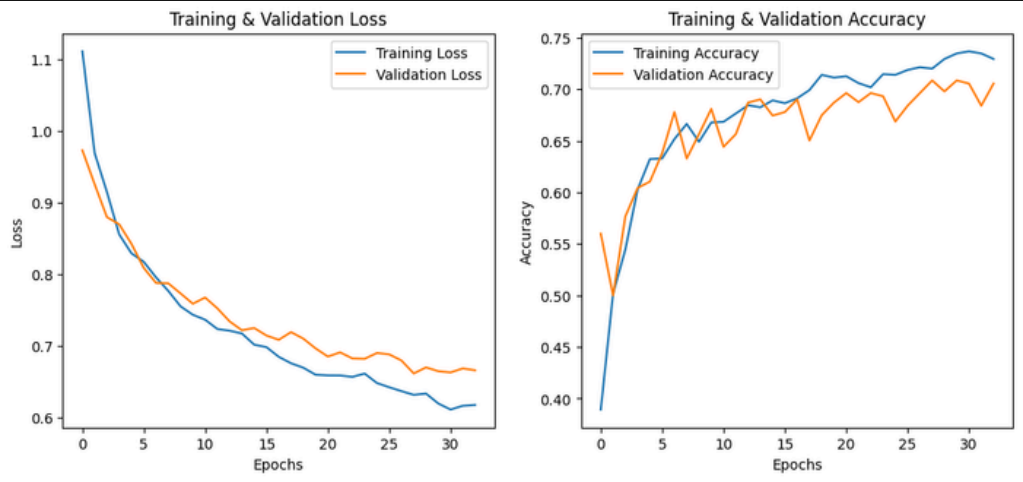
# Train InceptionV3
history_inception = model.fit(
    train_gen,
    validation_data=val_gen,
    steps_per_epoch=len(train_paths) // 64,
    validation_steps=len(val_paths) // 64,
    epochs=100,
    callbacks=[tf.keras.callbacks.EarlyStopping(
        monitor='val_loss', patience=5, restore_best_weights=True)]
)

# Save Model
model.save("/kaggle/working/InceptionV3_knee_osteo_model.keras")
```

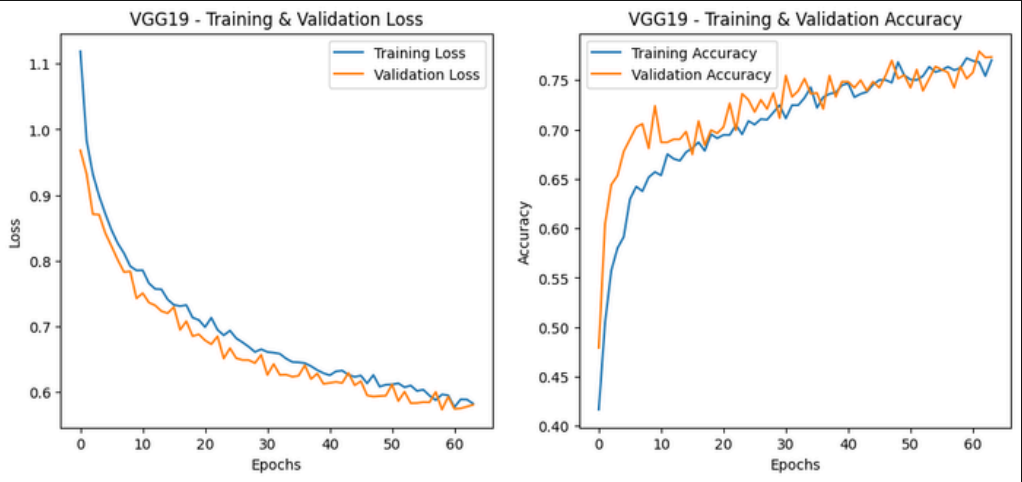
RESULTS



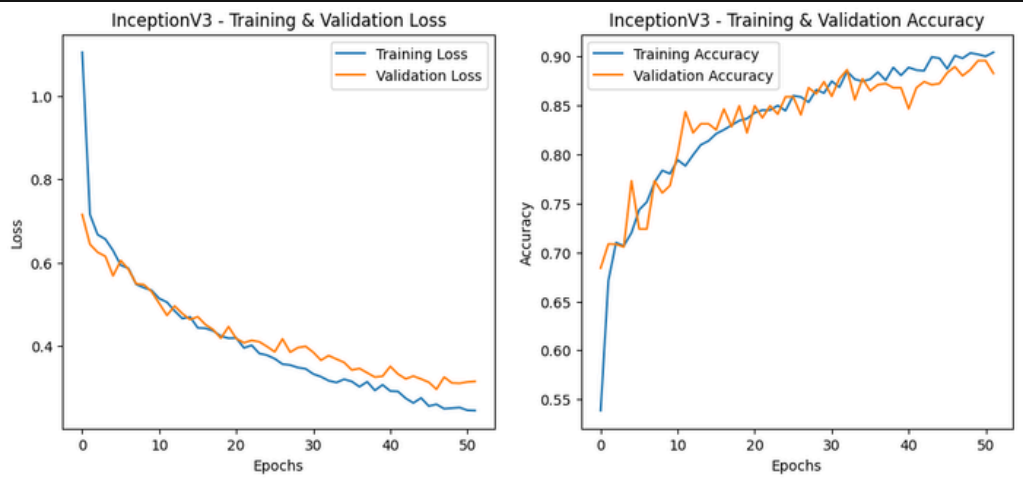
VGG16



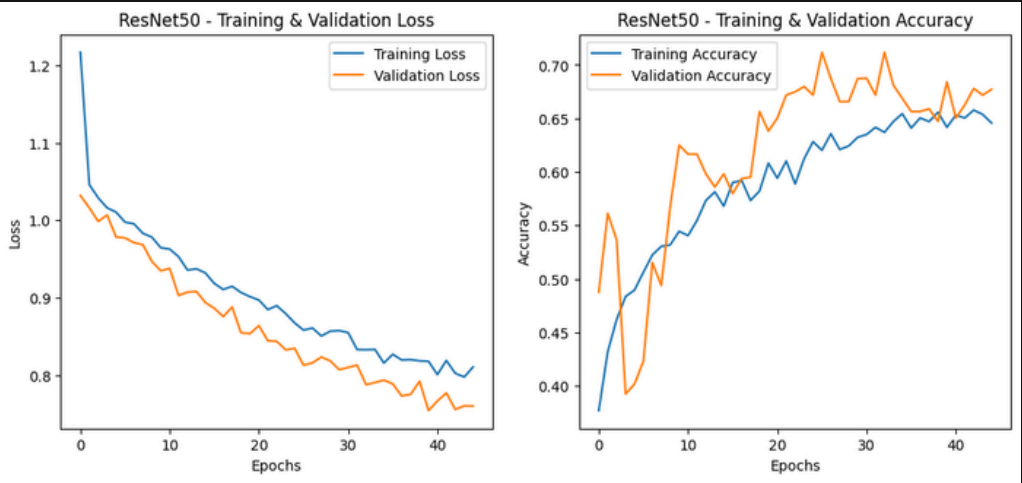
VGG19



InceptionV3



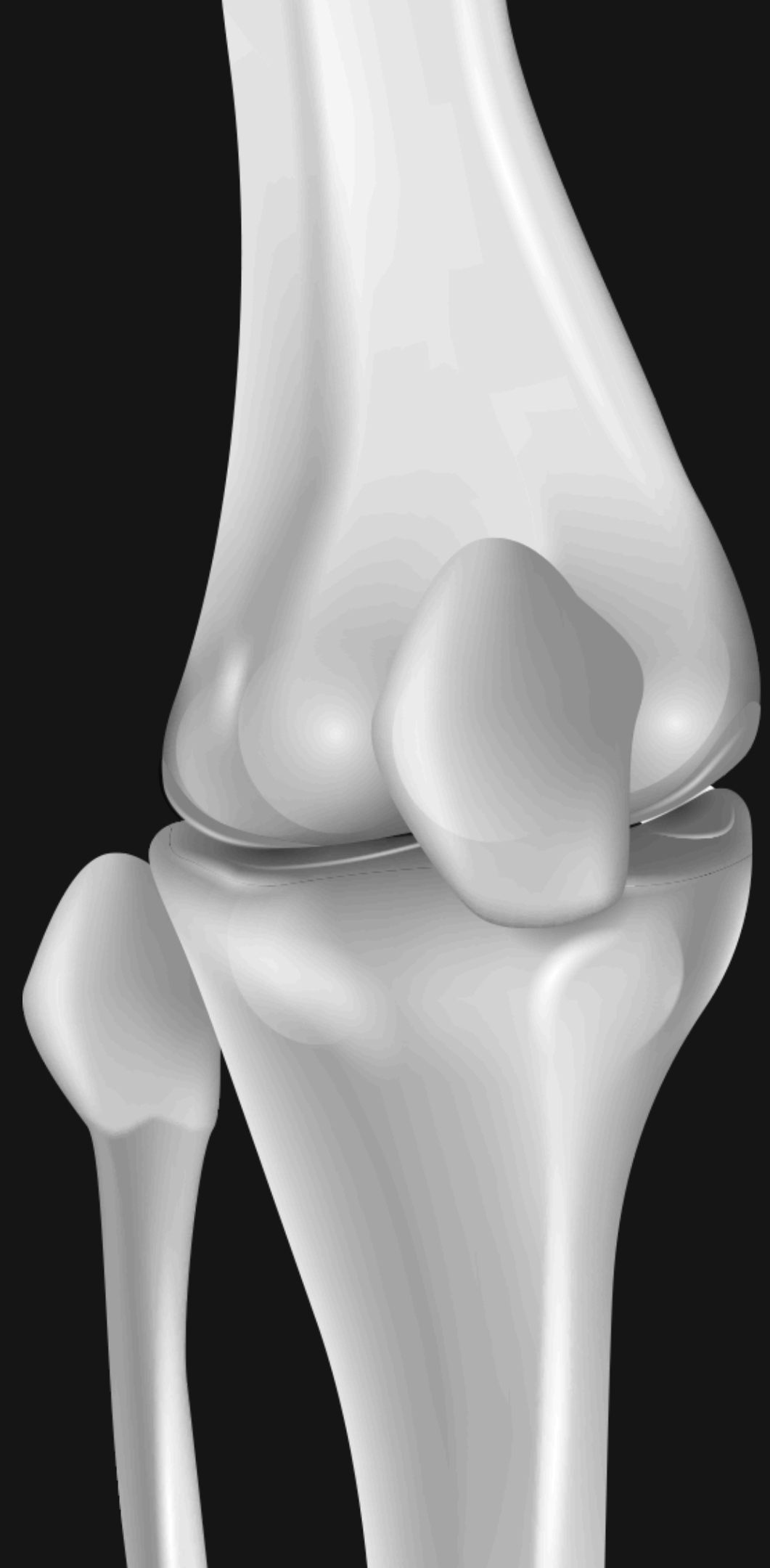
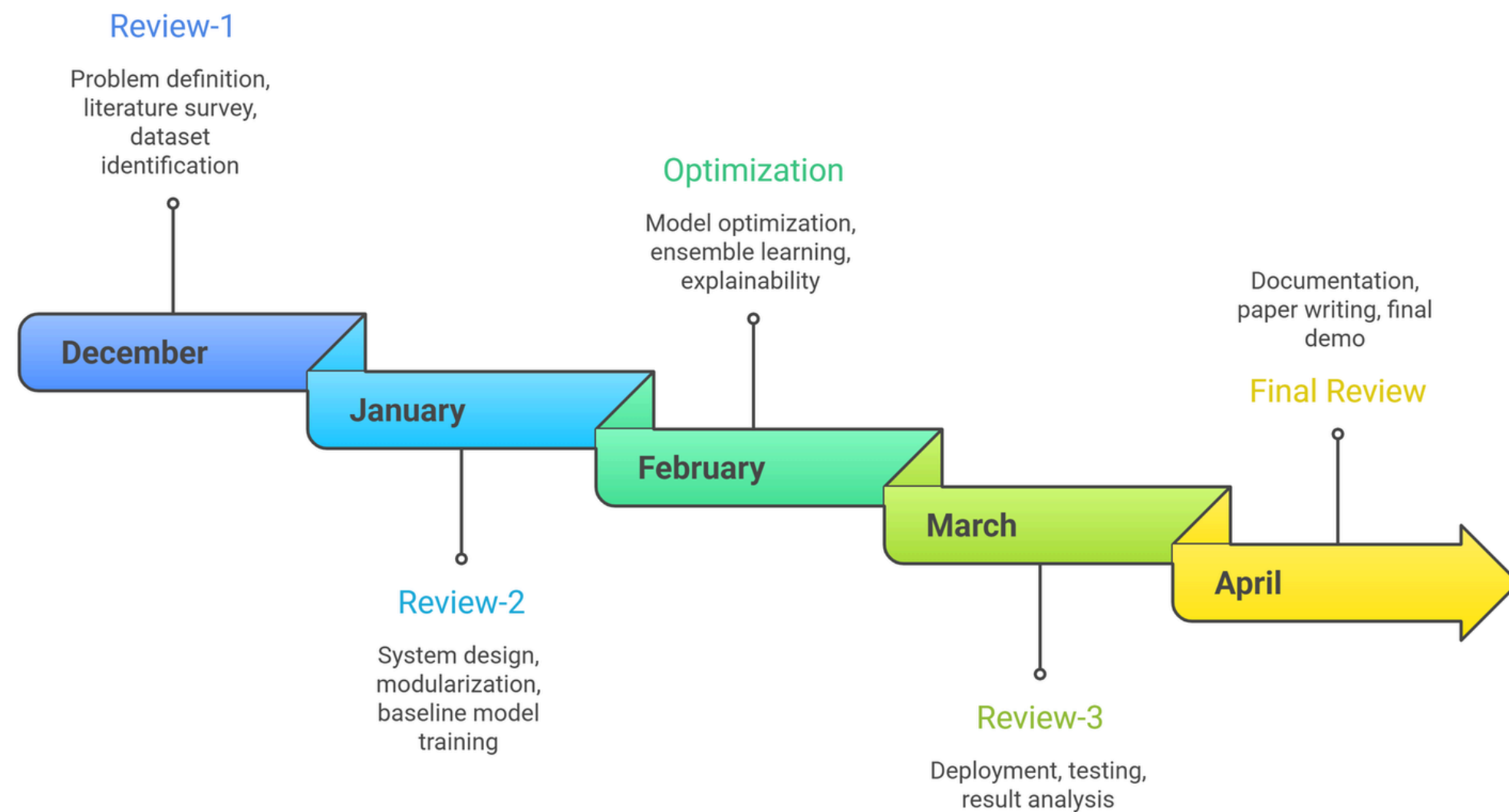
ResNet50



Model	Accuracy	Macro Precision	Macro Recall	Macro F1-Score	Weighted F1-Score
VGG16	0.7	0.75	0.67	0.69	0.7
VGG19	0.75	0.78	0.75	0.75	0.75
ResNet50	0.66	0.76	0.61	0.62	0.65
InceptionV3	0.88	0.88	0.88	0.88	0.88

PLANNING OF PROJECT

Project Timeline



TESTCASE SPECIFICATION

Test case specification defines the conditions under which the system is tested to verify correctness, reliability, and smooth interaction between different modules of the system.

Test Case ID	Module	Input	Expected Output
UT-01	Data Acquisition	Knee X-ray image	Image loaded successfully
UT-02	Preprocessing	Raw image	Resized & normalized image
UT-03	Data Augmentation	Training images	Augmented images with correct labels
UT-04	Model Prediction	Preprocessed image	Class label with confidence score
UT-05	Explainability	Model + image	Grad-CAM heatmap generated
UT-06	User Interface	Image upload	Image displayed correctly

UNIT TESTING

Unit testing ensures correctness of individual modules

Test Case ID	Integrated Modules	Input	Expected Output
IT-01	Preprocessing → Model → Output	X-ray image	Correct osteoporosis class
IT-02	Dataset → Training → Evaluation	Dataset	Trained model with metrics
IT-03	Model → Explainability	Test image	Prediction with heatmap
IT-04	UI → Backend Model	Uploaded image	Result shown on UI
IT-05	UI + Validation	Invalid image	Error message displayed

INTEGRATION TESTING

Integration testing ensures smooth end-to-end system operation

CONTRIBUTIONS

- Dataset collection and class-wise analysis
- Image resizing, normalization, and augmentation
- Train-validation-test split
- Ensured uniform preprocessing for all models

MODULE 1

Data Preprocessing & Dataset Preparation

- Implemented VGG16 pre-trained model
- Fine-tuned final layers
- Trained and validated the model

MODULE 2

Transfer Learning Model – VGG16

- Implemented VGG19 pre-trained model
- Fine-tuned the top classification layers
- Trained and validated the model

MODULE 3

Transfer Learning Model – VGG19

- Implemented ResNet50 architecture
- Model training and optimization
- Performance evaluation

MODULE 4

Transfer Learning Model – ResNet50

- Trained InceptionV3 model
- Feature extraction and classification
- Result analysis

MODULE 5

Transfer Learning Model – InceptionV3

THANK YOU